

Introduction to PThreads and Dining Philosophers

Introduction

- In this lab, we will learn how to utilize Pthreads to handle synchronization issues
- We'll first look at how to implement P_Threads and use them for counting
- Then we'll look at the Dining Philosopher problem a format of synchronization problems that are everywhere in Operating Systems and Software

Introduction

- Helpful man pages:
 - Pthreads(7): Overview of Pthreads programming model
 - `pthread_create`: Routine to create a new thread
 - `pthread_join`: One thread waits for another to terminate
 - `pthread_mutex_init`: Initialize a `pthread_mutex_t` mutex
 - `pthread_mutex_lock`: lock a mutex
 - Also presents `pthread_mutex_unlock` and `pthread_mutex_trylock`
- These functions are all you should need to make the code modifications required for the lab

Threads vs. Processes

- Multiple processes work well when the application tasks performed by each process are: (a) unrelated, or (b) require communication well suited to pipes or sockets
- More intimate, complex, or fine-grained collaboration among concurrent parts of an application is often difficult or impossible when multiple processes are used
- With an application's work divided among multiple processes, the context switch overhead generally decreases performance
- The thread model was invented to address these issues
 - Multiple threads sharing an address space decreases several sources of overhead

Threads vs. Processes

- Sharing of significant state information among processes is problematic because:
 - Multiple copies of state data are created or a shared segment must be declared and managed
 - Exchanging state information requires expensive IPC mechanisms and complete consistency among multiple copies is often not possible
 - Using semaphores among processes involves system call overhead
- Under the thread model context switch overhead is lower, all global variables are shared, and implementing mutexes is cheaper

Threads vs. Processes

- When you create a new process, an entire copy of the parent process' process control block is copied to the child
- When you create a new thread, only the components of the process control block that are *necessary to create a new thread of control* are actually allocated for the child
 - A new PC, registers image, user and kernel stack, and some other misc. info are allocated for the new thread.
 - The code and data regions of the address space are not copied, but are now shared between the parent and the child
- For these reasons, threads are often thought of as lightweight processes
 - Lower context switch and lower memory use compared to an equivalent set of processes

Thread Pros and Cons

- By allowing threads of control to share a common address space, the thread model provides an efficient way to multi-task a job over several threads of control
 - State information available to each thread is updated in a common copy without the use of IPC mechanisms
 - Context switching from thread to thread is more efficient than switching the context of an entire process because much of the address space remains the same
- These advantages, however, come with their own drawbacks:
 - The use of shared data must be synchronized among threads
 - Functions used by multiple threads of control must be *reentrant*
 - A reentrant function must not hold static data over successive calls or return a pointer to static data

Creating Threads

- In the POSIX API, the *fork()* system call creates processes
- Under Linux, *fork()* makes use of the *clone()* system call :
 - `int clone(int (*fn)(void *), void *child_stack, int flags, void *arg, ... )`
- *Clone()* provides a fine-grain interface for controlling what resources are shared among parent and child
 - The *flags* argument controls which resources are shared and which copied
 - For example, calling *clone()* with the CLONE_FILES bit set in the *flags* argument will create a child which shares the parent's open file table (as opposed to creating a copy of the parent's file descriptor table for the child)
 - Process creation uses flags that copy the parent address space
 - Thread creation uses flags that share the address space

Thread Libraries

- For many reasons, thread standards are often implemented and distributed as libraries
 - A common standard allows for portable software
 - Libraries are an efficient way of distributing widely used user level code
 - Depending on which metrics the programmer cares about, different user level designs of the thread system will yield much different performance
- In today's lab, we will use the Native POSIX Threads Library, which implements the POSIX Thread API standard for Linux

The POSIX Standard

- The POSIX standard specifies a set of interfaces (i.e. functions and header files) that can be used for threaded programming
- These interfaces require that the underlying implementations meet a certain set of criteria:
 - A single process may contain multiple threads, all of which execute the same program, i.e. share the same text segment
 - Threads share the same global memory (data and heap segments).
 - Each thread has its own stack (automatic variables)
- There are several other characteristics specified by the standard.
 - Threads share a Process ID, but have a unique Thread ID.
 - See the pthreads(7) manual page for a more complete description

ptcount.c

- Open ptcount.c and read through it to get a feel for the program's structure. As it is now, the program doesn't do much.
- The goal of this lab is to learn how PThreads work by modifying this program to create some number of threads executing the same routine
 - NUM_THREADS controls the number of threads created
- The routine each thread executes (inc_count) will spin in a loop (bound by the LOOP_BOUND command line arg), incrementing some global integer (by the scalar specified by the INCREMENT command line arg) for each iteration

pthread_create

- To create each pthread, use:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,  
                  void *(*start_routine)(void*), void *arg);
```

- The arguments are as follows:
 - **thread** is an address pointing to the allocated thread object, filled in on return
 - **attr** is a pointer to the attribute object for this thread. Attribute objects provide a mechanism for changing the configurable aspects of a thread.
 - **start_routine** is a pointer to the routine in which the thread should start its execution. For this lab, the start routine for all of our threads will be **inc_count**
 - **arg** is a pointer to the arguments passed to the start routine. Each thread `start_routine` is only allowed one argument. Thus, multiple arguments are packaged in a struct, whose address is passed as the argument.

pthread_create (cont.)

- The easiest way to create all of our threads is in a for loop
- In order to save time, we have provided the code you should use. Make sure you understand where to put it and what each of its arguments means:

```
for ( i = 0; i < NUM_THREADS; i++)  
    . . .  
    pthread_create(&threads[i], &attr, inc_count, (void *)targs);  
}
```

- When pthread_create returns, the thread described by &threads[i] are live (i.e. ready to run and waiting to be scheduled or has started running inc_count code already)

Waiting on the Threads

- You'll recall from the IPC lab that we used `waitpid` to have the parent process wait for its children to complete and exit

- We achieve similar behavior for threads with **`pthread_join`**:

```
int pthread_join(pthread_t thread, void **value_ptr);
```

- **`pthread_join`** will suspend execution of the calling process (or thread) until the target thread, the *thread* referred to by the `pthread_t` instance in argument *thread*, has terminated.
- **`value_ptr`** is a pointer to the data returned by the `pthread_exit()` library call. We do not use this data, so you can just pass in `NULL` here.
- Again, we use **`pthread_join`** in a for loop:

```
for (i = 0; i < NUM_THREADS; i++) {  
    pthread_join(threads[i], NULL);
```

Testing the Threaded Program

- At this point you have the parent creating NUM_THREADS threads and then waiting for the threads created to finish executing **inc_count**
- Now is a good time to go ahead and test the threaded program. To run the program, use:

```
-bash-3.2$ ./ptcount LOOP_BOUND INCREMENT
```

- Where LOOP_BOUND is the number of times the loop in each thread should iterate, and INCREMENT is the amount to increment count for each iteration.
- If you created and joined the PThreads correctly, you should see something like:

```
-bash-3.2$ ./ptcount 100 1
```

```
Thread: 0 finished. Counted: 100
```

```
Thread: 1 finished. Counted: 100
```

```
Thread: 2 finished. Counted: 100
```

```
Main(): Waited on 3 threads. Final value of count = 300. Done.
```

- Which is exactly what we wanted. Each thread increments the count 100 times (by 1 each time) and the final value of count is 300 because we have three threads.

Puzzling Behavior

- Though these initial tests seem to show that *ptcount* is working correctly, but running *ptcount* with a larger loop bound results in puzzling behavior
- The makefile has a target *test* runs *ptcount* with a large loop bound. The output should look something like this:

```
-bash-3.2$ make test
./ptcount 100000 1
Thread: 0 finished. Counted: 100000
Thread: 1 finished. Counted: 100000
Thread: 2 finished. Counted: 100000
Main(): Waited on 3 threads. Final value of count = 288284. Done.
```

- Each thread reports that it counts up to 100000, but the final count is significantly less than 300000. Note that in *ptcount.c* each thread prints out a *local* variable to report its own count.
- Furthermore, if you run the test again, the final count changes, but it is still not the expected result. What is going on?

Incrementing Count

- Each thread updates the count using the = and + operator in C:

```
count = count + my_args->inc;
```

- Observe that this one line instruction in C, is actually implemented by three instructions in the x86 hardware:

mov	\$(C_ADDR), %eax	; Move count into a register
add	%ebx, %eax	; Add inc to count
mov	%eax, \$(C_ADDR)	; Store count back into memory

- Now, recall that each thread is sharing only the global instance of *count*, which is loaded from memory and stored again each time any thread increments it
- While this does not explain observed behavior yet, it's necessary for understanding the solution

A Preemptive Scheduler

- Recall from lecture that the Linux scheduler is preemptive.
 - The scheduler has the ability to stop a running process, with the intention of resuming it later, and switch context to another process in the READY state.
- Now, imagine a scenario where the scheduler just happened to interleave two running threads in the following way:

T1	T2
1. mov \$(C_ADDR), %eax	
2.	mov \$(C_ADDR), %eax
3.	add %ebx, %eax
4.	mov %eax, \$(C_ADDR)
5. add %ebx, %eax	
6. mov %eax, \$(C_ADDR)	

An Interleaving Problem

- Observe that:
 - At time 1, T1 loads the count value from memory.
 - Next, T1 is preempted and T2 begins executing. In its execution, it loads the *same value* from memory. T2 continues, incrementing the value it had loaded, and stores it back into memory.
 - Now, at time 5, T2 is preempted by T1. At this time, T1 still holds the value it loaded *at time 1* into %eax. T1 proceeds to increment *this value*, and stores it back into memory.
 - When T1 stores its value of count back into memory, the work done by T2 is essentially lost.
- This explains why the value read by the parent process after the threads had finished incrementing is substantially lower than what you might expect
 - Each instance of such an interleaving loses an increment of count by a thread
- Probability of inconsistencies in shared data, such as *count*, depends on the length of the code section using it, the number of threads sharing it, and the number of times threads execute the code section operating on shared data.

Questions?

Fixing the Problem

- Regions of code that update shared data are known as *critical sections*.
- In order to ensure data shared among cooperating threads remains correct, we need a way of ensuring that only one thread may execute inside a critical section at a time:
 - This is called *mutual exclusion*
- Policy Goal:
 - All updates to shared data must be executed atomically with respect to other operations on that data.

pthread_mutex_lock

- Library calls for lock and unlock:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- The implementation of these routines permits only one thread to lock a given mutex at a time
 - *A mechanism* for mutual exclusion on critical section
- When a thread executes **pthread_mutex_lock** and the mutex is already locked, the operating system blocks the calling thread.
- All threads blocking on the mutex are awakened when the lock is released -> the owner calls **pthread_mutex_unlock**

Mutex Solution

- Place mutex lock/unlock around the critical section in `ptcount.c` to ensure mutually exclusive access to the *count* variable
 - Note that the code already declares an instance of `pthread_mutex_t count_mutex`
- When you have built *ptcount* with the locks in place, *ptcount* should produce the following output:

```
-bash-3.2$ make test
```

```
./ptcount 1000000 1
```

```
Thread: 0 finished. Counted: 1000000
```

```
Thread: 2 finished. Counted: 1000000
```

```
Thread: 1 finished. Counted: 1000000
```

```
Main(): Waited on 3 threads. Final value of count = 3000000. Done.
```

Atomic Add Solution

- The pthread mutex solution works and is portable but is quite expensive for a simple add operation.
- Far more efficient would be to use the GCC specific built-in function specifically for incrementing values:

`__atomic_add_fetch(type* ptr, type val, int memorder)`

- **type** is an overloaded integral or pointer type referring to memory sizes of 1, 2, 4, or 8 bytes. This is determined automatically by the compiler.
- **ptr** is a pointer to the field to increment
- **val** is the amount to add to the current value dereferenced by ptr
- **memorder** is a hint to the compiler about code motion in optimization. In this lab we will use the value `__ATOMIC_RELAXED` for this argument.
- [Documentation for GCC atomics](#)

Built-In Functions

- Built-in functions are functions provided by the compiler. They often times give access to an optimized set of assembly instructions that are normally hard to express from standard C code.
- The `__atomic_add_fetch()` built-in function gives access to the “lock add” x86 assembly instruction which performs the load, increment, and store atomically.
- Generally, a fallback implementation should be provided for compiling with other compilers that may not have the same built-in function. For GCC this is outlined in the following example:

```
#ifdef __GNUC__  
    __atomic_add_fetch(/* arguments */);  
#else  
    /* pthread mutex solution */  
#endif
```

Questions

Dining Philosophers

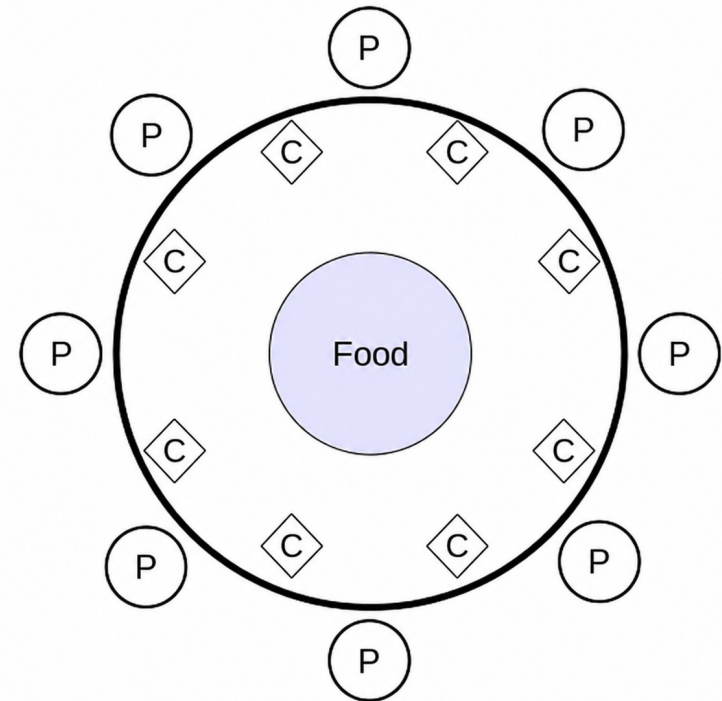
- The Dining Philosophers is a classic case study which highlights the solutions to common concurrency problems
 - Multiple threads using multiple resources
 - Different sets of resources used by different threads
 - Threads spend different amounts of time using resources and between intervals of resource use
 - Deadlock can occur because of a set of interactions among different threads and resources
- First proposed by Dijkstra (1965) as a problem of coordinating access by five computers to five tape drives
 - Retold in its more amusing current form by Hoare
- Few real-world problems map directly onto its structure
 - Many share characteristics: multiple threads, multiple resources, varied patterns of resource use

Dining Philosophers

- A set of philosophers spend their lives alternating between thinking and eating
- Philosophers sit around a table with a shared bowl of food
- To eat, philosophers must hold two implements
- Implements are placed on the table between philosophers
- Each philosopher this has a right and left implement
- Each philosopher uses a different set of resources
- Implements can only be acquired one at a time
- When a philosopher becomes hungry, she tries to pick up the left implement and then the right
- If an implement is missing, the philosopher waits for it to appear
- A hungry philosopher holding two implements eats until no longer hungry, puts down her implements and thinks

Dining Philosopher's Visual

- N philosophers, N forks
- Food has unrestricted concurrent access
- Chopsticks are exclusive use resources
- Each chopstick plays a different role for its philosophers (L/R)
- Each chopstick is used by a different set of philosophers
- Deadlock appears quite unlikely to happen
- Happens “quickly” in practice



Dining Philosopher Implementation

- Starter code implements the “classic” dining philosophers problem with its vulnerability to deadlock
- Starter code also contains some components labeled `ASYMMETRIC` and `WAITER` which are associated with two different approaches to a solution you will work on.

Dining Philosopher Implementation

- Code is a fairly straightforward implementation decomposed into a number of components
 - `dining_philosophers.c`
- Code begins with includes and defined constants
 - Constants are used to control many aspects of behavior
- Next, a definition of the philosopher structure
 - Note the *prog* and *prog_total* fields which track the number of times a philosopher has gone through the think-eat cycle during an accounting period and during program execution, respectively
- Next com some global variables:
 - *Diners*: array of philosopher structures
 - *Stop*: global stop flag
 - *chopstick*: array of mutexes representing the chopsticks

Dining Philosopher Implementation

- Global continued
 - *waiter*: mutex used to represent the waiter the waiter-based solution
 - *available_chopsticks*: array of integers used to represent chopstick availability in the waiter solution
- Next is a set of utility routines used in various solutions
 - Return pointers to philosopher to left and right of argument, chopstick to left and right, and pointer to available flag of left and right chopstick of a given philosopher
- *think_one_thought()* and *eat_one_mouthful()* routines
 - Used in *dp_thread()* routine to represent activity
- *dp_thread()* routine is code executed by each philosopher thread which implements the think-eat cycle until told to stop, and does accounting on how many cycles completed

Dining Philosopher Implementation

- *set_table()* routine initializes data structures representing chopsticks, initializes the philosopher structures and creates the philosopher threads
- *print_progress()* prints progress statistics for each philosopher, and zeroes the prog filed so progress during each accounting period is counted as well as the total
 - Five philosophers per line and a blank line between statistics for each accounting period
- *main()* calls *set_table()*, prints out a header, and falls into the accounting and deadlock detection loop
- Root thread zeroes philosopher period progress, then sleeps for ACCOUNTING_PERIOD seconds
 - Checks to see if any progress made while it slept
 - Infers deadlock if not, and sets Stop
 - Prints statistics in any case

Dining Philosopher Implementation

- Run the existing code
 - `bash> make dp_test`
- Your output should be similar, but remember thread behavior and deadlock are affected by many random factors
 - Context switches, other load on system, interrupts, etc

```
plato:starter_code$ make dp_test
```

```
gcc -g dining_philosophers.c -lpthread -lm -o dp
```

```
./dp
```

Dining Philosophers Update every 5 seconds

```
p0=    1012/1056  p1=        1/1  p2=    492/492  p3=    913/913  p4=        0/0
```

```
p0=      0/1056  p1=        0/1  p2=      0/492  p3=      0/913  p4=        0/0
```

Deadlock Detected

Questions?

Asymmetric Solution

- The output shows that deadlock occurred during the first accounting period, after threads had performed a variable number of think-eat cycles
 - “P1 = 123/456” entry indicates that P1 executed 123 think-eat cycles in the current accounting period and has 456 total
 - Numbers may not be completely consistent as there is no concurrency control between main and philosopher threads
 - Try running the test several times and see that behavior varies
- Deadlock occurs because each philosopher has picked up the left fork before any have pick up the right
- Asymmetric solution is to have the even numbered philosophers pick up in left-right order, while odd-numbered pick up in right-left order

Asymmetric Solution

- Make the necessary change to `dp_asymmetric.c`
 - test `me->id` for even or odd and alter mutex lock order accordingly
 - `bash> make dp_asymmetric_test`
- If your implementation is correct, then the program should run for 10 5-second cycles and complete without deadlock
- Note how many think-eat cycles each philosopher makes in each accounting cycle and total
 - This will vary with the platform (cycle4, 1005D-*, WSL, etc)
 - Was several hundred thousand on development machine
 - Ask yourself are all philosophers getting roughly equal time?
- Try running it a few more times and see how much behavior varies due to random chance and system context

Waiter Solution

- Now consider a slightly more complex solution using a Pthread condition variable approach
 - Mutex *waiter* represents a waiter in the cafe that will “give” the chopsticks to a philosopher as a **pair**
 - Note that this will constrain concurrency more than the asymmetric solution as this creates a region where only one philosopher at a time can obtain its chopsticks
 - Relevant changes are in *dp_thread()* code where philosophers obtain and give back their chopsticks
- This solution does not need the chopstick array of mutexes
 - Use the array of integers *available_chopsticks* instead, whose integrity will be protected by the waiter mutex, and condition variable programming pattern

Waiter Solution

- Get-chopsticks section ensures that testing `my_chopsticks_free` and `mark_my_chopsticks_free` set of operations are **ATOMIC** using *waiter*
- Free-chopsticks section uses waiter to ensures the `mark_my_chopsticks_free` and Signal sets of operations are done **ATOMICALLY**
- Consider types and pointers carefully as the helper routines return pointers to available flags and philosophers

```
pthread_mutex_lock(&waiter);
```

```
while (!( my_chopsticks_free )){  
    pthread_cond_wait(&(me → can_eat),&waiter);  
}
```

```
mark_my_chopsticks_taken;  
pthread_mutex_unlock(&waiter);
```

```
Eat;
```

```
pthread_mutex_lock(&waiter);
```

```
mark_my_chopstick_free;  
Signal those who might care they became free
```

```
pthread_mutex_unlock(&waiter);
```

Waiter Solution

- When your solution is complete and correct, your solution should produce output similar to the asymmetric solution
 - Runs through 10 cycles and completes without deadlock
- Note the number of think-eat cycles and ask yourself
 - Are these significantly higher, lower, or the same as the asymmetric solution?
 - Why are the number of the cycles that way?
- Another point of interest is the while loop testing the condition and calling `pthread_cond_wait()`
 - Why does this need to be a loop
 - Hint: Consider possible events between when the decision to send the signal is made and when the signal is received

Waiter Solution

- Does this solution prevent starvation?
 - Hint: No!!!
- Try to extend your solution to count the number of times a philosopher is awakened and both chopsticks are *not free*, so it must wait again
- Experiment with tests in the chopstick freeing area that send a signal to a philosopher only when both its chopsticks are free
 - You should find that a small but significant percentage of the time a chopstick is taken between when the signal is sent and when the receiving philosopher tries to get its chopsticks
- Consider what would happen in these retry cases if the while loop was an *if-then* instead

Conclusion

- PThreads provides a standard interface for multi-threaded programming available on many platforms
- Multi-threaded software architectures have many advantages over multi-process architectures but:
 - Concurrent access to shared data creates critical sections which can cause incorrect shared data
- Concurrency control in the form of mutex locks, also called semaphores, is the most common way to ensure correct multi-threaded programs
- Other types of concurrency control exist, with various advantages and disadvantages

Conclusion

- Creation, exposure and use of concurrency arises in many different situations
- Shared data access must be properly controlled for correct programs
- The PThreads mutex you have seen here is a classic example
- Principles of concurrent programming in PThreads also apply in a wide range of other situations such as with the dining philosophers

Conclusions

- The dining philosophers is a simple problem with a surprising number of subtle aspects
 - Deadlock seems extremely unlikely, yet happens quit quickly
 - Solutions are not all that difficult, but have different implications
- Plausible but incorrect solutions also easy to construct
- Shows that knowing if a solution is correct is also hard
- Neither of these solutions to preventing deadlock prevent starvation
- Consider how to implement the Waiter solution with a Monitor representing the waiter
 - Waiter can maintain a queue of requests, ensuring all philosophers eventually eat